

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Основы алгоритмизации и программирования

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему

ИГРОВОЕ ПРОГРАММНОЕ СРЕДСТВО РИТМИЧЕСКОГО
ВЗАИМОДЕЙСТВИЯ С МУЗЫКАЛЬНЫМИ КОМПОЗИЦИЯМИ

БГУИР КР 6-05-0612-01 106 ПЗ

Студент: гр. 451001 Држевецкий Н. А.

Руководитель:
асс. Шостак Е.В.

Минск 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРОТОТИПОВ, ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОЕКТИРУЕМОМУ ПРОГРАММНОМУ СРЕДСТВУ	7
1.1 Примеры решения аналогичных задач	7
1.2 Требования к проектируемому программному средству.....	7
2 АНАЛИЗ ТРЕБОВАНИЙ К ПС И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	10
2.1 Описание функциональности ПС.....	10
2.2 Спецификация функциональных требований	12
3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА	14
3.1 Разработка алгоритма ПС	14
3.2 Алгоритмы отдельных модулей	14
4 СОЗДАНИЕ (КОНСТРУИРОВАНИЕ) ПРОГРАММНОГО СРЕДСТВА	20
4.1 Структура программного средства	20
4.2 Описание логики, процедур и функций.....	21
5 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ	25
5.1 Тестирование программного средства.....	25
5.2 Экспериментальная проверка.....	27
5.3 Анализ полученных данных	27
5.4 Результаты тестирования.....	27
6 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ	28
6.1 Установка приложения “BeatKey”	28
6.2 Эксплуатация приложения “BeatKey”	29
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33
ПРИЛОЖЕНИЯ.....	34
ВЕДОМОСТЬ	52

ВВЕДЕНИЕ

Современные игровые технологии активно развиваются и охватывают широкий спектр задач, выходящих за рамки исключительно развлекательных функций. Особое место среди них занимают музыкальные и ритм-игры, которые способствуют развитию чувства ритма, реакции, концентрации внимания и координации действий пользователя. Подобные программные средства находят применение как в игровой индустрии, так и в образовательной среде, а также используются в качестве инструмента для самотренировки и творческого самовыражения.

Ритм-игры представляют собой жанр, в котором игровой процесс напрямую синхронизирован с музыкальным сопровождением. Пользователь должен выполнять определённые действия (нажатия клавиш или их удержание) в строго заданные моменты времени, соответствующие ритмической структуре музыкального произведения. Примерами популярных игр данного жанра являются *osu!*[1], *Friday Night Funkin'*[2] и *A Dance of Fire and Ice*[3], каждая из которых использует собственный подход к визуализации ритма и взаимодействию с игроком.

Целью данной курсовой работы является разработка игрового программного средства «BeatKey» — ритм-игры для персонального компьютера, ориентированной на нажатие клавиш в такт музыкальному сопровождению. Особенностью разрабатываемого приложения является возможность загрузки пользовательских музыкальных композиций и карт таймингов, а также наличие базового редакторского подхода к созданию и настройке ритмических последовательностей.

Программное средство реализовано на языке программирования C++ с использованием фреймворка Qt, который предоставляет инструменты для создания графического пользовательского интерфейса, обработки событий ввода, работы с мультимедиа и организации анимации. Использование Qt позволяет обеспечить переносимость приложения и стабильную работу с аудиофайлами и визуальными элементами игры.

В ходе выполнения курсовой работы были поставлены следующие задачи:

- 1 Анализ существующих ритм-игр и формирование требований к разрабатываемому программному средству.

- 2 Проектирование архитектуры приложения и логики игрового процесса.

- 3 Реализация механики воспроизведения музыки и синхронизации игровых объектов с тайм-кодами.

4 Разработка пользовательского интерфейса и системы визуальной обратной связи.

5 Тестирование работоспособности программы и анализ корректности игровых сценариев.

6 Подготовка документации, описывающей структуру, алгоритмы и результаты разработки.

Разработка данного программного средства представляет собой практическую задачу по созданию интерактивного приложения, сочетающего работу с графикой, звуком и пользовательским вводом в режиме реального времени. Результатом курсовой работы является полностью функционирующее игровое приложение, соответствующее поставленным требованиям и пригодное для дальнейшего расширения и использования.

1 АНАЛИЗ ПРОТОТИПОВ, ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОЕКТИРУЕМОМУ ПРОГРАММНОМУ СРЕДСТВУ

1.1 Примеры решения аналогичных задач

На сегодняшний день существует ряд программных продуктов и игр, реализующих механику ритмического взаимодействия пользователя с музыкальными композициями. Рассмотрим наиболее известные из них.

osu! — популярная ритм-игра, в которой игроку необходимо нажимать на игровые объекты в строго определённые моменты времени, синхронизированные с музыкальным сопровождением. Игра поддерживает загрузку пользовательских карт, содержащих тайминги нажатий, и предлагает различные режимы взаимодействия.

Достоинства:

- 1 Высокая точность синхронизации с музыкой.
- 2 Поддержка пользовательских карт и музыкальных файлов.
- 3 Большое сообщество и разнообразие контента.
- 4 Развитие реакции и чувства ритма.

Недостатки:

- 1 Высокий порог входа для новых пользователей.
- 2 Сложный интерфейс и система настроек.
- 3 Зависимость от интернет-сервисов для полноценного функционала.

Friday Night Funkin' — ритм-игра, в которой игрок должен нажимать клавиши в такт музыке, ориентируясь на визуальные подсказки в виде движущихся стрелок. Игра ориентирована на реакцию и запоминание ритмических паттернов.

Достоинства:

- 1 Простой и понятный игровой процесс
- 2 Низкий порог входа.
- 3 Яркая визуальная обратная связь.
- 4 Развитие координации и внимания.

Недостатки:

- 1 Ограниченные возможности по созданию собственных карт.
- 2 Отсутствие встроенного редактора таймингов.
- 3 Жёсткая привязка визуальных элементов к игровому стилю.

1.2 Требования к проектируемому программному средству

1.2.1 Назначение разработки.

Разрабатываемое программное средство BeatKey представляет собой локальную ритм-игру, предназначенную для взаимодействия пользователя с музыкальными композициями посредством нажатия клавиш в заданные моменты времени. Цель разработки — создание игрового приложения, позволяющего загружать музыкальные файлы и соответствующие им карты таймингов, а также оценивать точность действий пользователя.

1.2.2 Состав выполняемых функций.

- 1 Загрузка музыкальных файлов формата MP3 и WAV.
- 2 Загрузка карт таймингов из текстовых файлов.
- 3 Воспроизведение музыкальных композиций.
- 4 Отображение игровых объектов, синхронизированных с музыкой.
- 5 Обработка нажатий клавиш пользователя в реальном времени.
- 6 Обработка нажатий клавиш пользователя в реальном времени.
- 7 Подсчет очков и количества здоровья (НР).
- 8 Отображение текущего состояния игры и результатов.
- 9 Поддержка пользовательских настроек (громкость, бинды клавиш).

1.2.3 Входные данные.

- 1 Музыкальные файлы.
- 2 Файлы карт таймингов.
- 3 Нажатия клавиш пользователя.
- 4 Пользовательские настройки (громкость, клавиши управления).

1.2.4 Выходные данные.

- 1 Визуальное отображение игровых объектов.
- 2 Информация о точности нажатий.
- 3 Отображение текущего количества очков и НР.
- 4 Сообщения о начале и завершении игры.
- 5 Итоговый результат прохождения композиции.

1.2.5 Требования к временным характеристикам.

- 1 Запуск воспроизведения музыки — не более 1 секунды.
- 2 Обработка пользовательского ввода — в режиме реального времени.
- 3 Обновление визуальных элементов — не реже 60 кадров в секунду.
- 4 Подсчет результатов нажатия — не более 10 миллисекунд.

1.2.6 Требования к надежности.

- 1 Корректная обработка некорректных или поврежденных файлов.
- 2 Корректная обработка некорректных или поврежденных файлов.

3 Стабильная работа при повторных запусках игры.

4 Предотвращение зависаний при ошибках ввода.

1.2.7 Условия эксплуатации.

Программа предназначена для работы на персональном компьютере без необходимости подключения к сети Интернет.

Поддерживаемые операционные системы:

1 Windows 11 (64-bit).

2 Windows 10 (64-bit).

3 Windows 7 (64-bit).

Управление осуществляется с помощью клавиатуры. Графический интерфейс отображается в оконном режиме и адаптируется под размеры экрана.

2 АНАЛИЗ ТРЕБОВАНИЙ К ПС И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

2.1 Описание функциональности ПС

Всевозможные акторы:

2.1.1 Загрузка карты и запуск игры.

Акторы: пользователь, программа.

Цель: загрузка музыкальной композиции и соответствующей карты таймингов, подготовка к началу игры.

Предусловия: программа запущена, игра находится в основном меню.

Основной сценарий:

1 Пользователь выбирает пункт меню «Загрузить карту».

2 Пользователь указывает файл карты таймингов.

3 Программа автоматически определяет и загружает связанный музыкальный файл.

4 Программа проверяет корректность загруженных данных.

5 После успешной загрузки программа отображает информацию о карте и готовности к игре.

Альтернативные сценарии:

1 Файл карты отсутствует или поврежден — программа уведомляет пользователя об ошибке.

2 Музыкальный файл не найден — программа сообщает о невозможности запуска игры.

2.1.2 Подготовка и запуск игрового процесса.

Акторы: пользователь, программа.

Цель: подготовка игрока к началу ритмического взаимодействия.

Предусловия: карта и музыкальный файл успешно загружены.

Основной сценарий:

1 Пользователь выбирает пункт «Начать игру».

2 Программа запускает подготовительный этап.

3 На экране отображается игровая зона и элементы интерфейса.

4 Через заданную задержку начинается воспроизведение музыки и активируется игровой процесс.

Альтернативные сценарии:

1 Пользователь пытается начать игру без загруженной карты — программа выводит предупреждение.

2.1.3 Основной игровой процесс.

Актёры: пользователь, программа.

Цель: выполнение нажатий клавиш в такт музыкальной композиции.

Предусловия: игровой процесс запущен, музыка воспроизводится.

Основной сценарий:

1 Программа отображает игровые объекты, движущиеся в сторону зоны попадания.

2 Пользователь нажимает назначенные клавиши в момент совпадения объекта с зоной попадания.

3 Программа фиксирует время нажатия и сравнивает его с тайм-кодом ноты.

4 В зависимости от точности нажатия определяется результат (отлично, хорошо, допустимо, промах).

5 Программа обновляет счет, здоровье и визуальную обратную связь.

Альтернативные сценарии:

1 Пользователь нажимает клавишу вне допустимого временного интервала — фиксируется промах.

2 Пользователь не нажимает клавишу — нота автоматически засчитывается как промах.

2.1.4 Подсчет результатов и завершение игры.

Актёры: программа.

Цель: определение итогового результата прохождения композиции.

Предусловия: все ноты карты обработаны.

Основной сценарий:

1 Программа анализирует все зарегистрированные нажатия.

2 Итоговый результат отображается на экране.

3 Программа завершает воспроизведение музыки.

4 Пользователю предлагается повторить игру или загрузить другую карту.

Альтернативные сценарии:

1 Значение здоровья игрока достигает нуля — игра завершается досрочно с сообщением о поражении.

2.1.5 Изменение настроек.

Актёры: пользователь, программа.

Цель: настройка параметров управления и звука.

Предусловия: программа запущена.

Основной сценарий:

1 Пользователь открывает окно настроек.

2 Пользователь изменяет громкость музыки и звуковых эффектов.

3 Пользователь назначает клавиши управления.

4 Программа сохраняет настройки и применяет их к игровому процессу.

Альтернативные сценарии:

1 Пользователь назначает недопустимую клавишу — программа запрашивает повторный ввод.

2.2 Спецификация функциональных требований

2.2.1 Загрузка карты и запуск игры.

1 Пользователь должен иметь возможность загружать карты таймингов из текстовых файлов.

2 Программа должна автоматически загружать соответствующий музыкальный файл.

3 Программа должна проверять корректность входных данных.

4 При ошибке загрузки программа должна уведомлять пользователя.

2.2.2 Подготовка и запуск игрового процесса.

1 Программа должна запускать игру только при наличии загруженной карты.

2 Должен присутствовать подготовительный интервал перед началом воспроизведения музыки.

3 Программа должна отображать текущее состояние игры.

2.2.3 Основной игровой процесс.

1 Программа должна отображать игровые объекты, синхронизированные с музыкой.

2 Программа должна обрабатывать нажатия клавиш в режиме реального времени.

3 Точность нажатий должна оцениваться по заданным временным интервалам.

4 Программа должна учитывать промахи при отсутствии нажатия.

5 Программа должна воспроизводить звуковую и визуальную обратную связь.

2.2.4 Подсчет результатов и завершение игры.

1 Программа должна вести подсчет очков и уровня здоровья игрока.

2 При снижении уровня здоровья до нуля игра должна завершаться досрочно.

3 После завершения композиции программа должна отображать итоговый результат.

2.2.5 Настройки.

1 Программа должна предоставлять интерфейс для изменения громкости музыки.

2 Программа должна предоставлять возможность изменения громкости звуковых эффектов.

3 Программа должна позволять переназначать клавиши управления.

4 Настройки должны сохраняться между запусками программы.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

3.1 Разработка алгоритма ПС

Алгоритм работы программы можно представить в виде последовательности действий:

- 1 Запуск программы и инициализация основных компонентов.
- 2 Загрузка карты таймингов и музыкальной композиции.
- 3 Подготовка игрового процесса.
- 4 Основной игровой цикл.
- 5 Обработка нажатий клавиш и оценка точности.
- 6 Подсчет результатов и контроль состояния игрока.
- 7 Завершение игры и отображение итогов.

3.2 Алгоритмы отдельных модулей

3.2.1 Алгоритм загрузки карты и подготовки игры

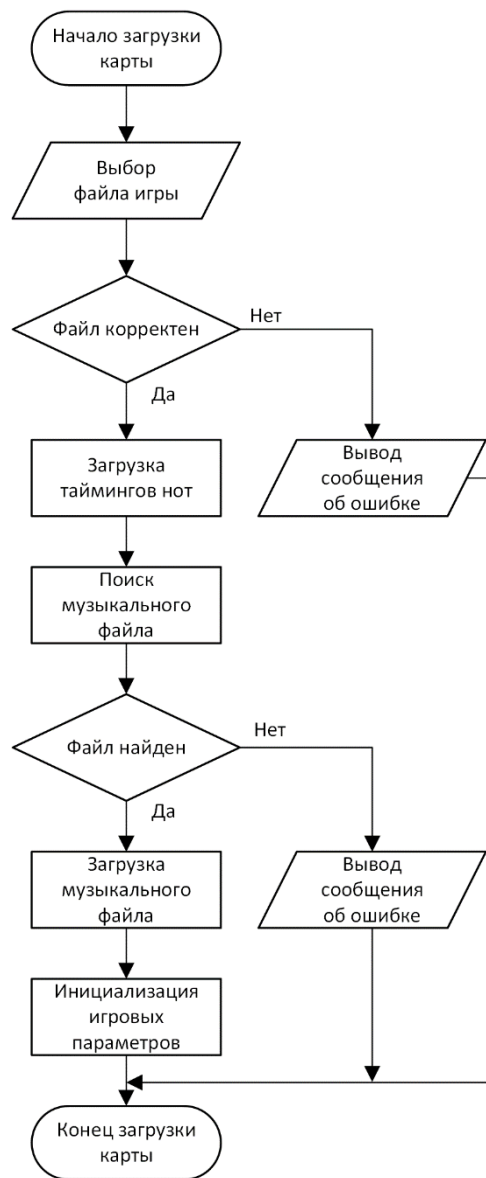


Рисунок 3.1 – Загрузка карты и подготовка игры

3.2.2 Алгоритм запуска и синхронизации игрового процесса

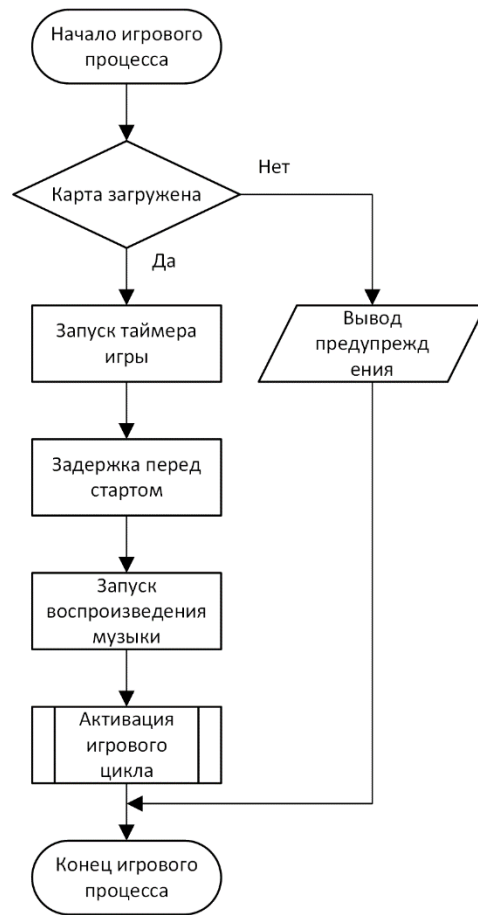


Рисунок 3.2 – Запуск и синхронизация игры

3.2.3 Алгоритм основного игрового цикла

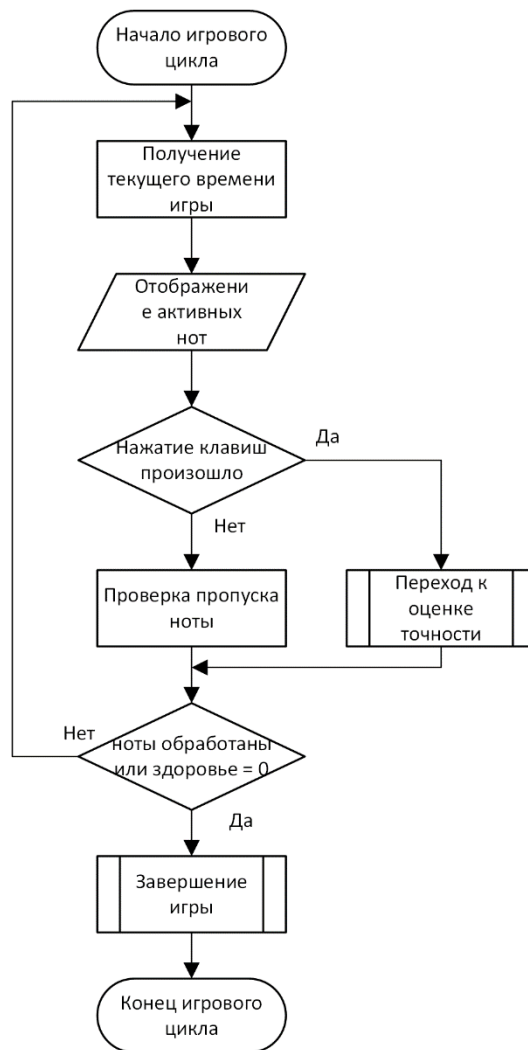


Рисунок 3.3 – Основной игровой цикл

3.2.4 Алгоритм оценки точности

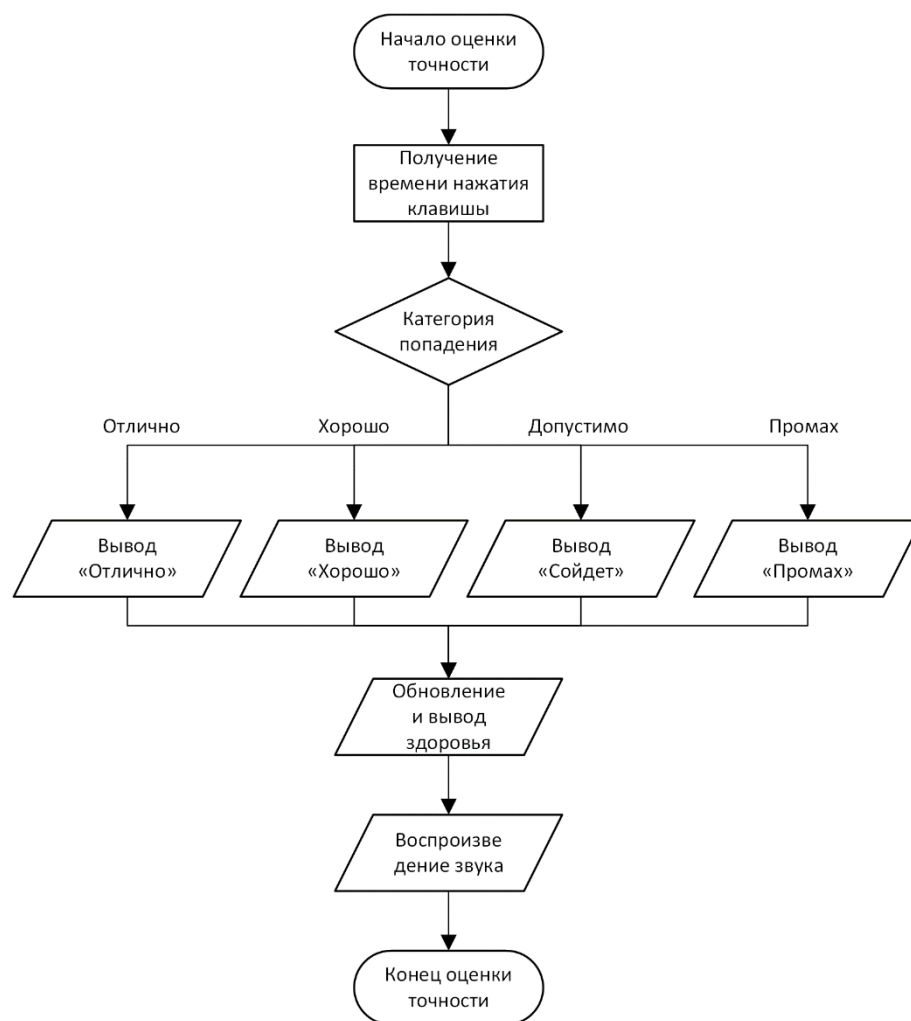


Рисунок 3.4 – Оценка попаданий

3.2.5 Алгоритм корректного завершения игры

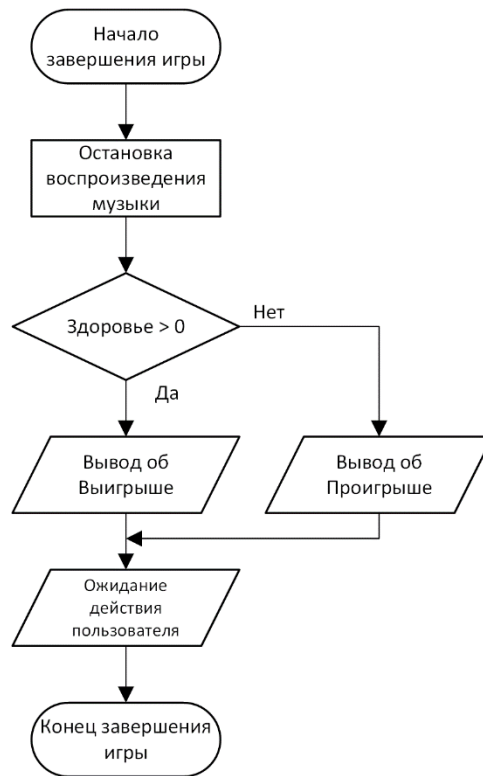


Рисунок 3.5 – Завершение игры

4 СОЗДАНИЕ (КОНСТРУИРОВАНИЕ) ПРОГРАММНОГО СРЕДСТВА

4.1 Структура программного средства

На основании разработанной архитектуры и алгоритмов реализуется программное средство «BeatKey». В процессе реализации осуществляется разработка всех основных модулей, их отладка и интеграция в единое приложение. Программное средство создаётся с использованием модульного подхода, что обеспечивает простоту сопровождения, возможность расширения функционала и повторного использования компонентов в будущем.

Архитектура приложения построена на модульной основе и включает следующие ключевые модули:

1 Модуль основного окна (MainWindow) – отвечает за управление интерфейсом пользователя, отображение игровых объектов, а также обработку событий клавиатуры. Этот модуль контролирует игровой цикл, синхронизацию нот с музыкальным сопровождением и визуализацию анимаций, таких как эффекты попаданий и промахов.

2 Модуль конфигурации игры (GameConfig) – обеспечивает хранение и загрузку настроек игры, включая громкость звуков (hit, miss, музыка) и пользовательские клавиши управления. Модуль реализован с использованием паттерна «синглтон», что гарантирует доступ к конфигурации из любого места программы и упрощает сохранение и изменение настроек.

3 Модуль диалога настроек (SettingsDialog) – предоставляет пользователю интерфейс для изменения параметров игры. Он включает слайдеры для регулировки громкости звуков и музыки, кнопки для изменения основных и дополнительных клавиш управления. Модуль также обрабатывает события ввода и обеспечивает обратную связь в реальном времени, обновляя текущие настройки при изменении пользователем.

4 Модуль работы с музыкой и звуковыми эффектами – реализует воспроизведение музыкальных треков и звуков hit/miss с корректной синхронизацией с игровым циклом. Модуль использует возможности фреймворка Qt (QMediaPlayer, QAudioOutput, QSoundEffect) для точного управления аудио и поддержки различных форматов файлов.

5 Модуль карты ритма (BeatMap) – отвечает за хранение тайм-кодов нот, их состояния (попадание, промах) и обработку игрового времени. Модуль обеспечивает возможность загрузки пользовательских карт из текстовых файлов и корректную синхронизацию нот с музыкальной дорожкой.

4.2 Описание логики, процедур и функций

4.2.1 Основные типы и структуры данных.

В модуле MainWindow определён основной тип данных Note, обозначающий игровую ноту. Он содержит следующие поля:

1 beatTime – момент времени (в миллисекундах), когда нота должна быть нажата.

2 hit – логическое значение, показывающее, была ли нота успешно сыграна.

3 missed – логическое значение, показывающее, была ли нота промахнута.

Также в модуле MainWindow определена структура BeatMap, содержащая:

1 musicPath – путь к музыкальному файлу карты;

2 notes – массив объектов Note, формирующий последовательность ритма.

В модуле GameConfig используется структура хранения настроек игры, включающая:

1 hitVolume, missVolume, musicVolume – значения громкости звуков hit, miss и фоновой музыки;

2 primaryKey, secondaryKey – коды клавиш управления;

3 m_settings – объект QSettings для сохранения и загрузки конфигурации из файла settings.ini.

В модуле SettingsDialog используются переменные primaryKeyCode и secondaryKeyCode для хранения временных значений клавиш, назначаемых пользователем через интерфейс диалога настроек.

4.2.2 Подпрограммы модуля MainWindow.

Таблица 4.1 – Описание процедур и функций модуля MainWindow

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
loadMap() ()	Загружает карту из текстового файла, создаёт объекты Note и синхронизирует с музыкальным файлом	void MainWindow:: loadMap()	–	–

Продолжение таблицы 4.1

startGame()	Запускает игровой цикл, сбрасывает состояние нот и HP	void MainWindow::startGame()	—	—
updateGame()	Обновляет состояние игры каждое тактовое событие таймера, запускает музыку, проверяет попадания и промахи	void MainWindow::updateGame()	—	—
keyPressEvent(QKeyEvent *event)	Обрабатывает нажатия клавиш, проверяет попадания в ноты, регистрирует результат	void MainWindow::keyPressEvent(QKeyEvent *event)	event	Содержит информацию о нажатой клавише
registerHit(qint64 diff)	Рассчитывает результат попадания (отлично, хорошо, сойдёт, промах), обновляет HP и отображает анимацию	void MainWindow::registerHit(qint64 diff)	diff	Разница между временем ноты и временем нажатия клавиши
animateScoreLabel()	Запускает анимацию изменения положения текста оценки	void MainWindow::animateScoreLabel()	—	—
updateHpLabel()	Обновляет текст и цвет HP	void MainWindow::updateHpLabel()	—	—
showSettingsDialog()	Отображает диалог настроек и передаёт текущие значения	void MainWindow::showSettingsDialog()	—	—
applySettings()	Применяет изменения из диалога настроек к игре	void MainWindow::applySettings()	—	—

4.2.3 Подпрограммы модуля SettingsDialog.

Таблица 4.2 – Описание процедур и функций модуля SettingsDialog

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
hitVolume()	Возвращает текущую громкость звука hit	float SettingsDialog::hitVolume()	—	—
missVolume()	Возвращает текущую громкость звука miss	float SettingsDialog::missVolume()	—	—
musicVolume()	Возвращает текущую громкость музыки	float SettingsDialog::musicVolume()	—	—
primaryKey()	Возвращает код основной клавиши	int SettingsDialog::primaryKey()	—	—
secondaryKey()	Возвращает код дополнительной клавиши	int SettingsDialog::secondaryKey()	—	—
setHitVolume(float volume)	Устанавливает громкость звука hit	void SettingsDialog::setHitVolume(float volume)	volume	Значение громкости от 0 до 1
setMissVolume(float volume)	Устанавливает громкость звука miss	void SettingsDialog::setMissVolume(float volume)	volume	Значение громкости от 0 до 1
setMusicVolume(float volume)	Устанавливает громкость музыки	void SettingsDialog::setMusicVolume(float volume)	volume	Значение громкости от 0 до 1
setPrimaryKey(int key)	Устанавливает основную клавишу управления	void SettingsDialog::setPrimaryKey(int key)	key	Код клавиши
setSecondaryKey(int key)	Устанавливает дополнительную клавишу управления	void SettingsDialog::setSecondaryKey(int key)	key	Код клавиши

Продолжение таблицы 4.2

keyPressEvent(QKeyEvent *event)	Обрабатывает нажатия клавиш для назначения биндов	void SettingsDialog::keyPressEvent(QKeyEvent *event)	event	Содержит информацию о нажатой клавише
onPrimaryKeyButtonClicked()	Переводит кнопку назначения основной клавиши в режим ожидания	void SettingsDialog::onPrimaryKeyButtonClicked()	—	—
onSecondaryKeyButtonClicked()	Переводит кнопку назначения дополнительной клавиши в режим ожидания	void SettingsDialog::onSecondaryKeyButtonClicked()	—	—
updateKeyButtonText(QPushButton *button, int key)	Обновляет текст кнопки, отображающий текущую клавишу	void SettingsDialog::updateKeyButtonText(QPushButton *button, int key)	button, key	Кнопка интерфейса и код клавиши

5 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

5.1 Тестирование программного средства

Тестирование проводилось вручную и частично автоматизировано в среде исполнения приложения. Были сформированы сценарии, имитирующие действия пользователей в различных условиях, включая стандартные и экстремальные ситуации. Ниже приведены примеры тестов, проведённых для проверки основных функций BeatKey.

Таблица 5.1 – Примеры проведённых тестов

№	Описание теста	Входные данные	Ожидаемый результат	Фактический результат
1	Загрузка корректной карты	Текстовый файл с 10 нот, mp3 рядом	Карта загружена, отображаются все ноты	Успешно
2	Загрузка карты без mp3	Текстовый файл без mp3	Появляется сообщение «MP3 не найден рядом с картой»	Успешно
3	Попытка начать игру без карты	—	Статус: «Карта не загружена», игра не запускается	Успешно
4	Нажатие основной клавиши в пределах точного попадания	primaryKey = Space, нота в зоне ± 25 мс	Отображается «ОТЛИЧНО», НР увеличивается	Успешно
5	Нажатие дополнительной клавиши в пределах попадания	secondaryKey = X, нота в зоне ± 50 мс	Отображается «ХОРОШО», НР корректно изменяется	Успешно
6	Нажатие клавиши вне диапазона времени ноты	Любая клавиша, нота ещё не в зоне	Попадание не регистрируется	Успешно
7	Проммах ноты	Нота не нажата до конца $\pm \text{missAfterTime}$	Отображается «ПРОМАХ», НР уменьшается	Успешно
8	Перегрузка слайдеров настроек	hitVolume = 1.5, musicVolume = -0.5	Значения корректируются в диапазоне 0–1	Успешно

Продолжение таблицы 5.1

9	Изменение клавиш управления через диалог настроек	primaryKey = Z, secondaryKey = A	Клавиши успешно изменены, игра реагирует на новые кнопки	Успешно
10	Закрытие диалога настроек без применения	Нажатие Cancel	Настройки не изменяются	Успешно
11	Воспроизведение музыки после старта игры	Карта с mp3	Музыка запускается через preStartDelay	Успешно
12	Поведение при низком НР	НР = 0	Игра останавливается, выводится «Вы проиграли»	Успешно
13	Анимация очков	Попадание «ХОРОШО»	label scoreLabel анимируется вверх и возвращается	Успешно
14	Перемещение окна и ресайз	Изменение размера окна	label scoreLabel и hpLabel корректно позиционируются	Успешно
15	Несколько нот одновременно	3 ноты близко по времени	Каждое попадание регистрируется корректно	Успешно
16	Попытка нажатия клавиши во время паузы	—	Нажатие игнорируется	Успешно
17	Сохранение и загрузка настроек	Изменение громкости и клавиш, перезапуск	Настройки сохраняются и загружаются корректно	Успешно
18	Ввод некорректного пути к карте	Пустой путь или не txt	Появляется предупреждение	Успешно
19	Быстрая последовательность нажатий	Множество нажатий клавиш подряд	Все попадания и промахи корректно регистрируются	Успешно
20	Проверка работы без музыки	Удалён mp3 файл	Игра запускается, но музыка отсутствует, ошибки нет	Успешно

5.2 Экспериментальная проверка

Для проверки адекватности игрового процесса программа была протестирована в реальных условиях: с участием пользователей разного возраста и уровня опыта. Результаты показали:

1 Игра полностью воспроизводима и стабильно реагирует на действия игрока.

2 Все этапы игрового процесса (загрузка карты, ожидание старта, игра, подсчёт очков) понятны и не вызывают затруднений.

3 Механика попадания, оценка точности и работа с НР интуитивно воспринимаются.

4 Победа определяется корректно на основании количества очков и состояния НР.

5.3 Анализ полученных данных

Все ключевые функции программного средства показали стабильную работу. Были зафиксированы следующие важные результаты:

1 Обработка нот и музыкального трека синхронизирована и точна, попадания регистрируются корректно.

2 Настройки громкости и клавиш сохраняются и загружаются корректно, даже после перезапуска программы.

3 Анимации и позиционирование элементов интерфейса работают плавно, не вызывая артефактов.

4 Программа устойчива к ошибкам ввода и некорректным действиям пользователя.

5.4 Результаты тестирования

Тестирование подтвердило соответствие программного средства требованиям, изложенным в техническом задании. Все ключевые функции реализованы и функционируют корректно. Программа устойчива к ошибкам ввода и некорректным действиям игрока. Интерфейс остаётся отзывчивым и информативным на всех этапах игры. Были выявлены незначительные UX-моменты (например, отсутствие визуального выделения активной ноты при приближении к hit-зоне), которые были устранены в процессе отладки.

Таким образом, можно считать, что BeatKey прошло успешную проверку и готово к использованию в учебной, демонстрационной или развлекательной среде.

6 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ

6.1 Установка приложения “BeatKey”

Для загрузки программного обеспечения необходимо перейти по предоставленной ссылке[4]. Она направляет на официальную страницу проекта на платформе GitHub.

На данной странице требуется найти и скачать архив программы в формате ZIP:

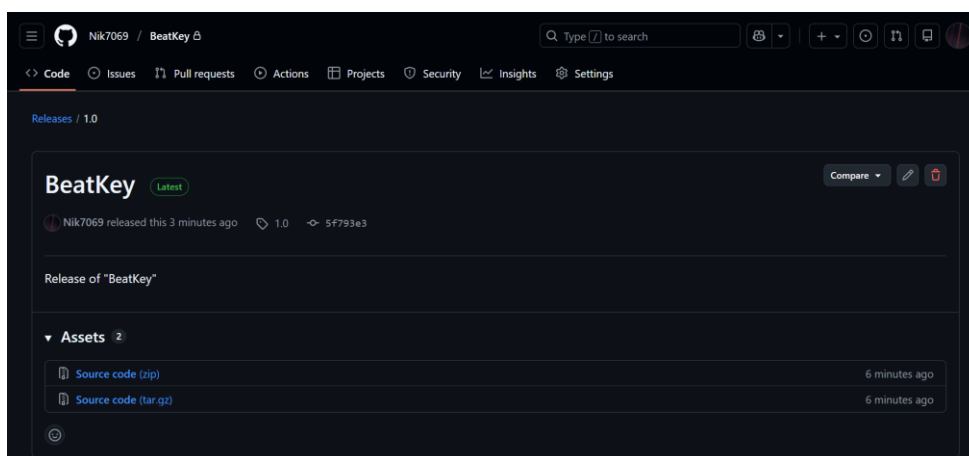


Рисунок 6.1 – Страница для скачивания приложения

После завершения загрузки ZIP-файла его необходимо распаковать в выбранную директорию. В результате разархивации должна быть создана папка, содержащая следующие файлы:

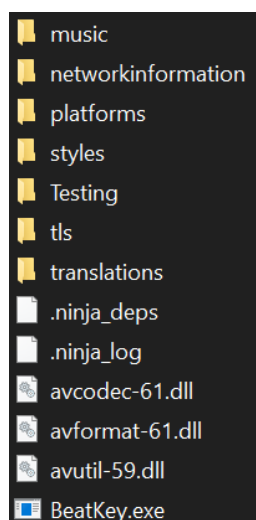


Рисунок 6.2 – Примерные файлы программы

Для начала работы с приложением следует открыть распакованную папку и запустить исполняемый файл "BeatKey.exe". После успешного запуска программа будет готова к эксплуатации.

6.2 Эксплуатация приложения “BeatKey”

Программа BeatKey рассчитана на использование на одном устройстве (например, ноутбуке, настольном ПК или интерактивной панели) и подходит для игры одного пользователя или нескольких участников по очереди. Подключение к Интернету не требуется.

В главном меню располагаются кнопки «Загрузить карту», «Начать игру». При запуске игры пользователь:

- 1 Загружает предварительно подготовленную карту с нотами и музыкальным треком.

- 2 Выбирает или оставляет стандартные клавиши управления (primaryKey и secondaryKey).

- 3 Следует правилам: вовремя нажимать клавиши, соответствующие нотам, и избегать промахов, чтобы сохранять НР.

Цель игры – набрать максимальное количество очков, точно попадая в ноты, не теряя НР. Игровой процесс условно делится на несколько этапов.

В этапе подготовки и загрузки карты:

- 1 Игрок выбирает файл карты (.txt) с нотами.

- 2 Программа автоматически ищет музыкальный трек (.mp3) с таким же именем в той же папке.

- 3 Пользователь может изменить настройки громкости и клавиш через меню «Звуки и бинды».

- 4 После загрузки карты отображается статус «Карта загружена», можно переходить к следующему этапу.

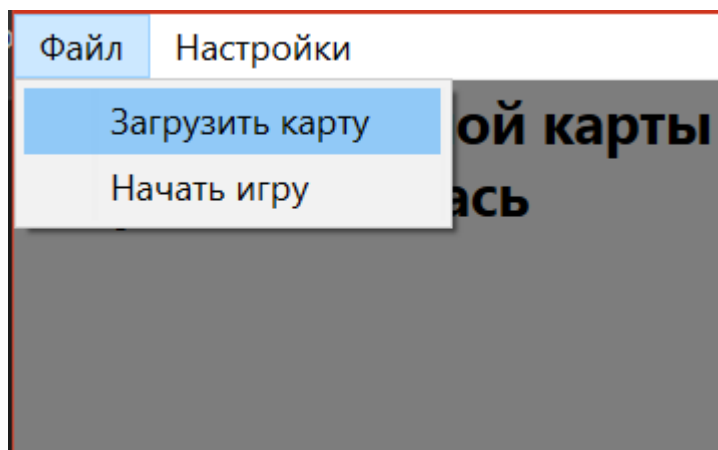


Рисунок 6.3 – Окно загрузки карты

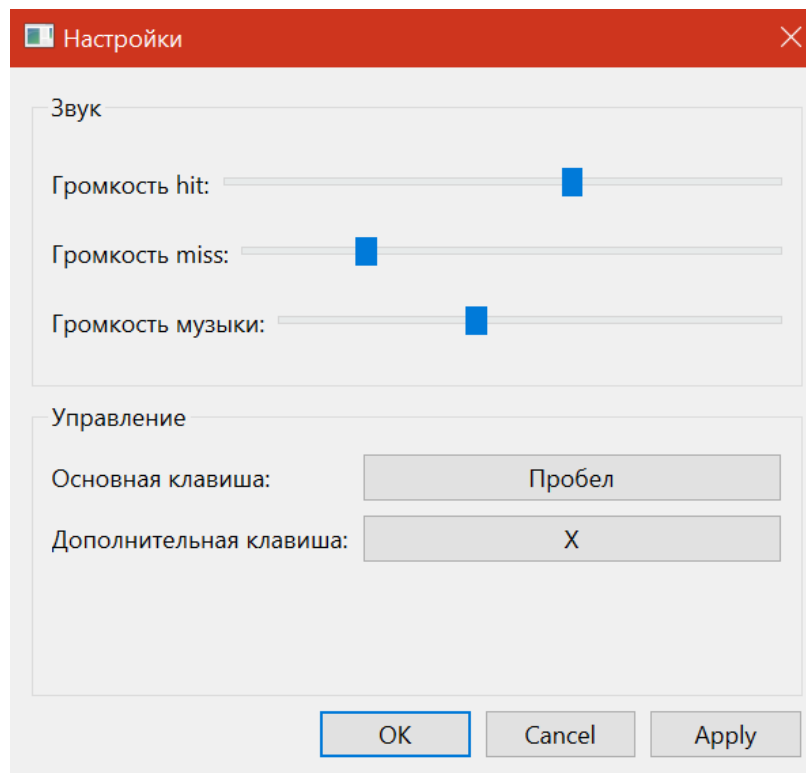


Рисунок 6.4 – Окно настроек звука и управления

На этапе игрового процесса:

1 Игра начинает отсчет времени, давая игроку возможность подготовиться.

2 Ноты начинают двигаться к hit-зоне справа налево.

3 Игрок должен нажимать клавиши `primaryKey` или `secondaryKey`, когда нота находится в зоне попадания.

4 Программа отображает текстовые оценки попаданий: «ОТЛИЧНО», «ХОРОШО», «СОЙДЕТ» или «ПРОМАХ».

5 HP изменяется в зависимости от точности попаданий; если HP достигает 0, игра завершается.

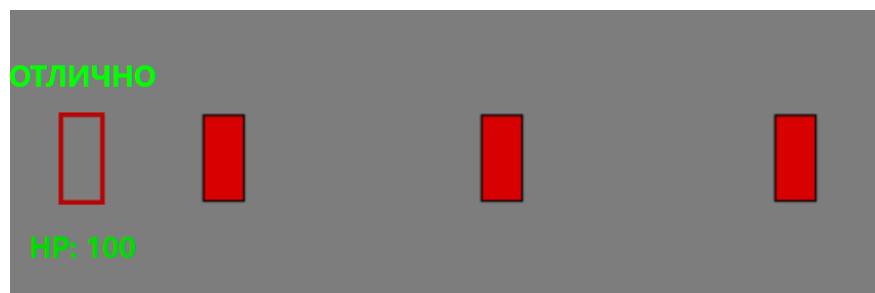


Рисунок 6.5 – Основное игровое окно

В этапе завершения игры:

- 1 Игра автоматически заканчивается после прохождения всех нот.
- 2 Программа выводит финальные результаты: последняя точность попаданий и оставшийся HP.
- 3 Игрок может повторить игру или вернуться в главное меню.

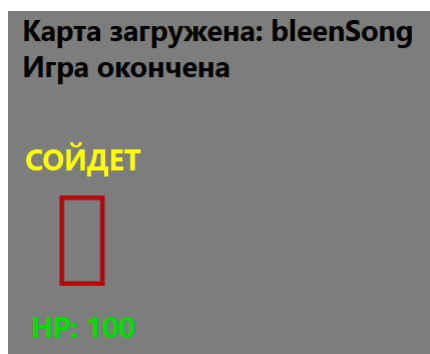


Рисунок 6.6 – Завершение игры и отображение результатов

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была спроектирована и разработана игра BeatKey, основанная на механике ритм-игр с точным попаданием в ноты, адаптированная для использования на одном устройстве без подключения к сети. Программное средство реализовано на языке C++ с использованием библиотеки Qt, что позволило эффективно организовать интерфейс, управлять аудиовыходом, анимацией нот и отображением очков, а также реализовать все необходимые этапы игрового процесса – от загрузки карты и музыкального трека до отслеживания попаданий, изменения НР и подсчета результатов.

В процессе работы были выполнены следующие задачи:

1 Проведен анализ аналогичных ритм-игр и сформулированы требования к собственной реализации, включая поддержку оффлайн-режима и пользовательских настроек клавиш и звука.

2 Описана функциональность программы и построены модели основных сценариев использования, таких как загрузка карты, игровая сессия и завершение игры.

3 Спроектированы алгоритмы обработки нот, синхронизации с музыкой, вычисления точности попаданий и отображения анимаций с использованием модульного подхода.

4 Реализован программный код с использованием компонентов Qt: QMediaPlayer, QAudioOutput, QSoundEffect, а также визуальных элементов для hit-зоны, анимации очков и НР.

5 Проведено всестороннее тестирование программы в различных сценариях, включая корректность работы с разными картами, настройками громкости и клавиш, что подтвердило стабильность и соответствие заявленным функциональным требованиям.

Разработка BeatKey представляет собой не только реализацию игрового механизма, основанного на точности и ритме, но и практическую задачу по проектированию, созданию и проверке комплексного пользовательского программного продукта. Результатом работы является полностью функционирующее приложение, пригодное для использования в учебной, демонстрационной или развлекательной среде, обеспечивающее интерактивный и наглядный игровой опыт.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

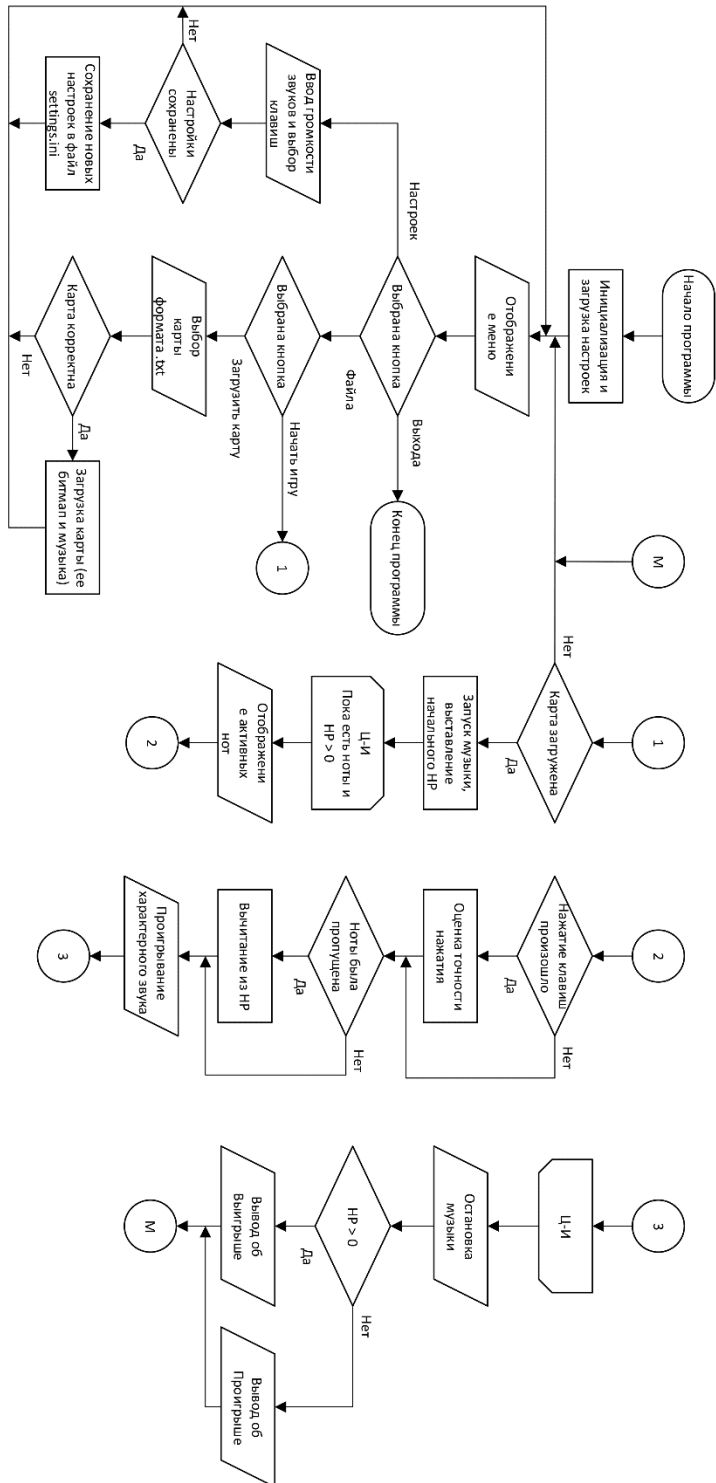
[1] osu! [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://osu.ppy.sh>.

[2] Friday Night Funkin' [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://ninja-muffin24.itch.io/funkin>.

[3] A Dance of Fire and Ice [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://fizzd.itch.io/a-dance-of-fire-and-ice>.

[4] Страница проекта “BeatKey” на платформе GitHub [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/Nik7069/BeatKey/releases/latest>.

ПРИЛОЖЕНИЕ А

[illegible]

ПРИЛОЖЕНИЕ Б

main.cpp

```
#include "mainwindow.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}
```

mainwindow.cpp

```
#include "mainwindow.h"
#include "../ui_mainwindow.h"

#include <QFileDialog>
#include <QKeyEvent>
#include <QDebug>
#include <QPainter>
#include <QPen>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setUpUi(this);

    // Загружаем настройки
    config.loadSettings();
    primaryKey = config.primaryKey();
    secondaryKey = config.secondaryKey();

    player = new QMediaPlayer(this);
    audioOutput = new QAudioOutput(this);

    // Устанавливаем громкость из настроек
    audioOutput->setVolume(config.musicVolume());
    player->setAudioOutput(audioOutput);

    // Подключаем действия меню
    connect(ui->actionLoadMap, &QAction::triggered,
            this, &MainWindow::loadMap);
    connect(ui->actionPlayGame, &QAction::triggered,
            this, &MainWindow::startGame);

    // Добавляем новое действие для настроек (нужно добавить в UI)
    // В дизайнере добавьте QAction в меню "Файл" с именем actionSettings
    connect(ui->actionSettings, &QAction::triggered,
            this, &MainWindow::showSettingsDialog);

    gameLoopTimer = new QTimer(this);
    connect(gameLoopTimer, &QTimer::timeout, this, &MainWindow::updateGame);
    gameLoopTimer->start(16);

    QPixmap rawNote("assets/note.png");
    QPixmap rawHitZone("assets/hitzone.png");
```

```

notePixmap = rawNote.scaled(
    hitZoneSize, hitZoneSize,
    Qt::KeepAspectRatio,
    Qt::SmoothTransformation
);

hitZonePixmap = rawHitZone.scaled(
    squareSize, squareSize,
    Qt::KeepAspectRatio,
    Qt::SmoothTransformation
);

if (notePixmap.isNull())
    qDebug() << "Failed to load note.png";
if (hitZonePixmap.isNull())
    qDebug() << "Failed to load hitzone.png";

hitSound.setSource(QUrl::fromLocalFile("assets/hit.wav"));
hitSound.setVolume(config.hitVolume());

missSound.setSource(QUrl::fromLocalFile("assets/miss.wav"));
missSound.setVolume(config.missVolume());

hitSound.setLoopCount(1);
missSound.setLoopCount(1);

scoreAnim = new QPropertyAnimation(ui->scoreLabel, "pos", this);
scoreAnim->setDuration(150);
scoreAnim->setEasingCurve(QEasingCurve::OutCubic);

scoreLabelBasePos = ui->scoreLabel->pos();
}

MainWindow::~MainWindow()
{
    config.saveSettings();
    delete ui;
}

void MainWindow::keyPressEvent(QKeyEvent *event)
{
    if (gameState != GameState::Playing)
        return;

    // Используем сохраненные клавиши
    int key = event->key();
    if ((key != primaryKey && key != secondaryKey) || event->isAutoRepeat())
        return;

    qint64 gameTime = gameTimer.elapsed();
    qint64 musicTime = gameTime - preStartDelay;

    Note *bestNote = nullptr;
    qint64 bestDiff = LLONG_MAX;
    qint64 diff;

    for (Note &note : currentMap.notes) {
        if (note.hit || note.missed)
            continue;

        diff = musicTime - note.beatTime;

        if (qAbs(diff) < qAbs(bestDiff)) {

```

```

        bestDiff = diff;
        bestNote = &note;
    }
}

qDebug() << musicTime;

if (!bestNote)
    return;

registerHit(bestDiff);

if (qAbs(bestDiff) <= missAfterTime) {
    bestNote->hit = true;
}
}

void MainWindow::loadMap()
{
    QString txtPath = QFileDialog::getOpenFileName(
        this,
        "Load beatmap",
        "",
        "Beatmap (*.txt)"
    );

    if (txtPath.isEmpty())
        return;

    QFile file(txtPath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        ui->statusMapLabel->setText("Ошибка загрузки карты");
        return;
    }

    currentMap.notes.clear();

    QTextStream in(&file);
    while (!in.atEnd()) {
        QString line = in.readLine();
        bool ok;
        qint64 t = line.toLongLong(&ok);
        if (ok)
            currentMap.notes.append(Note{ t, false, false });
    }

    file.close();

    // Ищем mp3 рядом
    QFileInfo info(txtPath);
    QString mp3Path = info.absolutePath() + "/" +
        info.completeBaseName() + ".mp3";

    if (!QFile::exists(mp3Path)) {
        ui->statusMapLabel->setText("MP3 не найден рядом с картой");
        return;
    }
}

```

```

    }

    currentMap.musicPath = mp3Path;
    player->setSource(QUrl::fromLocalFile(mp3Path));

    ui->statusMapLabel->setText("Карта загружена: " + info.completeBaseName());
}

void MainWindow::startGame()
{
    if (currentMap.notes.isEmpty()) {
        ui->statusGameLabel->setText("Карта не загружена");
        return;
    }

    // СБРОС СОСТОЯНИЯ НОТ
    for (Note &note : currentMap.notes) {
        note.hit = false;
        note.missed = false;
    }

    hp = 50;
    updateHpLabel();

    gameState = GameState::Playing;
    ui->statusGameLabel->setText("Подготовка...");

    player->stop();
    player->setPosition(0);

    gameTimer.restart();
    setFocus();
}

void MainWindow::updateGame()
{
    if (gameState != GameState::Playing)
        return;

    qint64 gameTime = gameTimer.elapsed();
    qint64 musicTime = gameTime - preStartDelay;

    // запуск музыки после подготовки
    if (gameTime >= preStartDelay &&
        player->playbackState() != QMediaPlayer::PlayingState) {

        player->play();
        ui->statusGameLabel->setText("Игра началась");
    }

    // авто-промах для непрожатых нот
    for (Note &note : currentMap.notes) {

        if (note.hit || note.missed)
            continue;

        if (musicTime > note.beatTime + 200) {
            note.missed = true;
            registerHit(9999); // промах
        }
    }
}

```

```

    }

    // конец игры (все ноты обработаны)
    bool allProcessed = true;
    for (const Note &note : currentMap.notes) {
        if (!note.hit && !note.missed) {
            allProcessed = false;
            break;
        }
    }

    if (allProcessed && musicTime > 0) {
        gameState = GameState::Idle;
        player->stop();
        ui->statusGameLabel->setText("Игра окончена");
        // ui->scoreLabel->setText("");
        // ui->hpLabel->setText("");
        update();
        return;
    }

    update(); // перерисовка
}

void MainWindow::paintEvent(QPaintEvent *)
{
    QPainter painter(this);
    painter.setRenderHint(QPainter::SmoothPixmapTransform, true);

    // фон
    painter.fillRect(rect(), QColor(125, 125, 125));

    // hit-зона
    int hitY = height() / 2 - hitZonePixmap.height() / 2;
    painter.drawPixmap(hitZoneX, hitY, hitZonePixmap);

    if (gameState != GameState::Playing)
        return;

    // ВРЕМЯ
    qint64 gameTime = gameTimer.elapsed(); // с начала игры
    qint64 musicTime = gameTime - preStartDelay; // с начала музыки (может быть < 0)

    int startX = hitZoneX + noteTravelDistance;
    int y = height() / 2 - notePixmap.height() / 2;

    // НОТЫ
    for (const Note &note : std::as_const(currentMap.notes)) {

        if (note.hit || note.missed)
            continue;

        qint64 beatTime = note.beatTime;

        // когда нота должна начать ехать (относительно музыки)
        qint64 spawnTime = beatTime - travelTime;

        // сколько она уже едет
        qint64 delta = musicTime - spawnTime;

        // ещё не появилась

```

```

        if (delta < 0)
            continue;

        // уже улетела
        if (delta > travelTime + missAfterTime)
            continue;

        double progress = double(delta) / double(travelTime);

        int x = startX - int(noteTravelDistance * progress);

        painter.drawPixmap(x, y, notePixmap);
    }
}

void MainWindow::registerHit(qint64 diff)
{
    diff = qAbs(diff);

    int hpDelta = 0;

    if (diff <= 25) {
        lastHitText = "ОТЛИЧНО";
        lastHitColor = QColor(0, 255, 0);
        hpDelta = +2;
    }
    else if (diff <= 50) {
        lastHitText = "ХОРОШО";
        lastHitColor = QColor(0, 200, 255);
        hpDelta = +1;
    }
    else if (diff <= missAfterTime) {
        lastHitText = "СОЙДЕТ";
        lastHitColor = QColor(255, 255, 0);
        hpDelta = 0;
    }
    else {
        lastHitText = "ПЛОХО";
        lastHitColor = QColor(255, 0, 0);
        hpDelta = -5;
    }

    hp += hpDelta;
    updateHpLabel();

    if (hpDelta >= 0)
        hitSound.play();
    else
        missSound.play();

    ui->scoreLabel->setText(lastHitText);
    ui->scoreLabel->setStyleSheet(
        QString("color: %1").arg(lastHitColor.name())
    );

    animateScoreLabel();

    // ПРОИГРЫШ
    if (hp <= 0) {
        gameState = GameState::Idle;
    }
}

```

```

        player->stop();
        ui->statusGameLabel->setText("Вы проиграли");
        update();
    }
}

void MainWindow::animateScoreLabel()
{
    scoreAnim->stop();

    scoreAnim->setStartValue(scoreLabelBasePos + QPoint(0, 10));
    scoreAnim->setEndValue(scoreLabelBasePos);

    scoreAnim->start();
}

void MainWindow::resizeEvent(QResizeEvent *)
{
    // X — центр hit-зоны
    int hitCenterX = hitZoneX + hitZonePixmap.width() / 2;

    // позиция scoreLabel
    int x = hitCenterX - ui->scoreLabel->width() / 2;

    int hitY = height() / 2 - hitZonePixmap.height() / 2;
    int y = hitY - ui->scoreLabel->height() - 20; // отступ сверху

    ui->scoreLabel->move(x, y);

    // сохраняем базовую позицию для анимации
    scoreLabelBasePos = ui->scoreLabel->pos();

    hitCenterX = hitZoneX + hitZonePixmap.width() / 2;
    hitY = height() / 2 - hitZonePixmap.height() / 2;

    x = hitCenterX - ui->hpLabel->width() / 2;
    y = hitY + hitZonePixmap.height() - 20;

    ui->hpLabel->move(x, y);
}

void MainWindow::updateHpLabel()
{
    hp = qBound(0, hp, 100);

    ui->hpLabel->setText(QString("HP: %1").arg(hp));

    QColor color;
    if (hp > 60)
        color = QColor(0, 220, 0);
    else if (hp > 30)
        color = QColor(255, 200, 0);
    else
        color = QColor(255, 0, 0);

    ui->hpLabel->setStyleSheet(
        QString("color: %1; font-weight: bold;").arg(color.name())
    );
}

```

```

void MainWindow::showSettingsDialog()
{
    if (!settingsDialog) {
        settingsDialog = new SettingsDialog(this);
        connect(settingsDialog, &SettingsDialog::settingsChanged,
            this, &MainWindow::applySettings);
    }

    // Устанавливаем текущие значения
    settingsDialog->setHitVolume(config.hitVolume());
    settingsDialog->setMissVolume(config.missVolume());
    settingsDialog->setMusicVolume(config.musicVolume());
    settingsDialog->setPrimaryKey(config.primaryKey());
    settingsDialog->setSecondaryKey(config.secondaryKey());

    settingsDialog->exec();
}

void MainWindow::applySettings()
{
    if (!settingsDialog)
        return;

    // Сохраняем в конфиг
    config.setHitVolume(settingsDialog->hitVolume());
    config.setMissVolume(settingsDialog->missVolume());
    config.setMusicVolume(settingsDialog->musicVolume());
    config.setPrimaryKey(settingsDialog->primaryKey());
    config.setSecondaryKey(settingsDialog->secondaryKey());

    // Применяем в игре
    hitSound.setVolume(config.hitVolume());
    missSound.setVolume(config.missVolume());
    audioOutput->setVolume(config.musicVolume());

    // Обновляем клавиши
    primaryKey = config.primaryKey();
    secondaryKey = config.secondaryKey();

    // Сохраняем настройки
    config.saveSettings();
}

```

mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QMediaPlayer>
#include <QAudioOutput>
#include <QElapsedTimer>
#include <QTimer>
#include <QKeyEvent>
#include <QSoundEffect>
#include <QPropertyAnimation>

// Добавляем новый заголовок
#include "settingsdialog.h"
#include "gameconfig.h"

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;

```



```

}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

protected:
    void keyPressEvent(QKeyEvent *event) override;
    void paintEvent(QPaintEvent *event) override;

private:
    Ui::MainWindow *ui;
    QMediaPlayer *player;
    QAudioOutput *audioOutput;

    // Добавляем диалог настроек
    SettingsDialog *settingsDialog = nullptr;
    GameConfig &config = GameConfig::instance();

    enum class GameState {
        Idle,
        Playing
    };

    GameState gameState = GameState::Idle;

    struct Note {
        qint64 beatTime; // когда должна быть нажата
        bool hit = false;
        bool missed = false;
    };

    struct BeatMap {
        QString musicPath;
        QVector<Note> notes;
    };

    BeatMap currentMap;

    QElapsedTimer gameTimer;
    const qint64 preStartDelay = 1000;

    QTimer *gameLoopTimer;

    const int hitZoneX = 20;
    const int hitZoneSize = 60;
    const int squareSize = 60;

    QPixmap notePixmap;
    QPixmap hitZonePixmap;

    const qint64 travelTime = 1000;
    const int noteTravelDistance = 600;
    const qint64 missAfterTime = 100;

    QString lastHitText = "";
    QColor lastHitColor = Qt::white;

```

```

QSoundEffect hitSound;
QSoundEffect missSound;

QPropertyAnimation *scoreAnim = nullptr;
QPoint scoreLabelBasePos;
int hp = 50;

// Добавляем клавиши
int primaryKey = Qt::Key_Space;
int secondaryKey = Qt::Key_X;

private slots:
    void loadMap();
    void startGame();
    void updateGame();
    void registerHit(qint64 diff);
    void animateScoreLabel();
    void resizeEvent(QResizeEvent *event) override;
    void updateHpLabel();

    // Добавляем новый слот для настроек
    void showSettingsDialog();
    void applySettings();
};

#endif // MAINWINDOW_H

```

gameconfig.cpp

```

#include "gameconfig.h"
#include <QDebug>

GameConfig::GameConfig()
: m_settings(QCoreApplication::applicationDirPath() + "/settings.ini",
             QSettings::IniFormat)
{
    loadSettings();
}

void GameConfig::loadSettings()
{
    m_hitVolume = m_settings.value("Sound/hitVolume", 0.4f).toFloat();
    m_missVolume = m_settings.value("Sound/missVolume", 0.4f).toFloat();
    m_musicVolume = m_settings.value("Sound/musicVolume", 0.5f).toFloat();
    m_primaryKey = m_settings.value("Keys/primaryKey", Qt::Key_Space).toInt();
    m_secondaryKey = m_settings.value("Keys/secondaryKey", Qt::Key_X).toInt();

    qDebug() << "Settings loaded:"
               << "hitVol:" << m_hitVolume
               << "missVol:" << m_missVolume
               << "musicVol:" << m_musicVolume
               << "primaryKey:" << m_primaryKey
               << "secondaryKey:" << m_secondaryKey;
}

void GameConfig::saveSettings()
{
    m_settings.setValue("Sound/hitVolume", m_hitVolume);
    m_settings.setValue("Sound/missVolume", m_missVolume);
    m_settings.setValue("Sound/musicVolume", m_musicVolume);
    m_settings.setValue("Keys/primaryKey", m_primaryKey);
    m_settings.setValue("Keys/secondaryKey", m_secondaryKey);

    m_settings.sync();
}

```

```

        qDebug() << "Settings saved";
    }

void GameConfig::setHitVolume(float volume)
{
    m_hitVolume = qBound(0.0f, volume, 1.0f);
}

void GameConfig::setMissVolume(float volume)
{
    m_missVolume = qBound(0.0f, volume, 1.0f);
}

void GameConfig::setMusicVolume(float volume)
{
    m_musicVolume = qBound(0.0f, volume, 1.0f);
}

void GameConfig::setPrimaryKey(int key)
{
    m_primaryKey = key;
}

void GameConfig::setSecondaryKey(int key)
{
    m_secondaryKey = key;
}

```

gameconfig.h

```

#ifndef GAMECONFIG_H
#define GAMECONFIG_H

#include <QSettings>
#include <QCoreApplication>

class GameConfig
{
public:
    static GameConfig& instance()
    {
        static GameConfig config;
        return config;
    }

    // Геттеры
    float hitVolume() const { return m_hitVolume; }
    float missVolume() const { return m_missVolume; }
    float musicVolume() const { return m_musicVolume; }
    int primaryKey() const { return m_primaryKey; }
    int secondaryKey() const { return m_secondaryKey; }

    // Сеттеры
    void setHitVolume(float volume);
    void setMissVolume(float volume);
    void setMusicVolume(float volume);
    void setPrimaryKey(int key);
    void setSecondaryKey(int key);

    // Загрузка и сохранение
    void loadSettings();
    void saveSettings();

private:

```

```

GameConfig();

float m_hitVolume = 0.4f;
float m_missVolume = 0.4f;
float m_musicVolume = 0.5f;
int m_primaryKey = Qt::Key_Space;
int m_secondaryKey = Qt::Key_X;

QSettings m_settings;
};

#endif // GAMECONFIG_H

```

settingsdialog.cpp

```

#include "settingsdialog.h"
#include "ui_settingsdialog.h"
#include <QKeyEvent>
#include <QDebug>
#include <QPushButton>
#include <QTimer>
#include <QDialogButtonBox>

SettingsDialog::SettingsDialog(QWidget *parent)
: QDialog(parent)
, ui(new Ui::SettingsDialog)
{
    ui->setupUi(this);

    // Настройка диапазонов слайдеров
    ui->hitVolumeSlider->setRange(0, 100);
    ui->missVolumeSlider->setRange(0, 100);
    ui->musicVolumeSlider->setRange(0, 100);

    // Важно: отключаем стандартную обработку пробела для кнопок
    ui->primaryKeyButton->setAutoDefault(false);
    ui->primaryKeyButton->setDefault(false);
    ui->secondaryKeyButton->setAutoDefault(false);
    ui->secondaryKeyButton->setDefault(false);

    // Подключение сигналов для кнопок клавиш
    connect(ui->primaryKeyButton, &QPushButton::clicked,
            this, &SettingsDialog::onPrimaryKeyButtonClicked);
    connect(ui->secondaryKeyButton, &QPushButton::clicked,
            this, &SettingsDialog::onSecondaryKeyButtonClicked);

    // Находим кнопки buttonBox и настраиваем
    QPushButton *applyButton = ui->buttonBox->button(QDialogButtonBox::Apply);
    QPushButton *okButton = ui->buttonBox->button(QDialogButtonBox::Ok);
    QPushButton *cancelButton = ui->buttonBox->button(QDialogButtonBox::Cancel);

    if (applyButton) {
        applyButton->setAutoDefault(false);
        applyButton->setDefault(false);
        connect(applyButton, &QPushButton::clicked, this, [this]() {
            emit settingsChanged();
        });
    }

    if (okButton) {
        okButton->setAutoDefault(false);
        okButton->setDefault(false);
        connect(okButton, &QPushButton::clicked, this, [this]() {
            emit settingsChanged();
        });
    }
}

```

```

        accept();
    });
}

if (cancelButton) {
    cancelButton->setAutoDefault(false);
    cancelButton->setDefault(false);
    connect(cancelButton, &QPushButton::clicked, this, &QDialog::reject);
}

// Обновляем текст кнопок
updateKeyButtonText(ui->primaryKeyButton, primaryKeyCode);
updateKeyButtonText(ui->secondaryKeyButton, secondaryKeyCode);
}

SettingsDialog::~SettingsDialog()
{
    delete ui;
}

float SettingsDialog::hitVolume() const
{
    return ui->hitVolumeSlider->value() / 100.0f;
}

float SettingsDialog::missVolume() const
{
    return ui->missVolumeSlider->value() / 100.0f;
}

float SettingsDialog::musicVolume() const
{
    return ui->musicVolumeSlider->value() / 100.0f;
}

int SettingsDialog::primaryKey() const
{
    return primaryKeyCode;
}

int SettingsDialog::secondaryKey() const
{
    return secondaryKeyCode;
}

void SettingsDialog::setHitVolume(float volume)
{
    ui->hitVolumeSlider->setValue(static_cast<int>(volume * 100));
}

void SettingsDialog::setMissVolume(float volume)
{
    ui->missVolumeSlider->setValue(static_cast<int>(volume * 100));
}

void SettingsDialog::setMusicVolume(float volume)
{
    ui->musicVolumeSlider->setValue(static_cast<int>(volume * 100));
}

void SettingsDialog::setPrimaryKey(int key)
{
    primaryKeyCode = key;
}

```

```

    updateKeyButtonText(ui->primaryKeyButton, key);
}

void SettingsDialog::setSecondaryKey(int key)
{
    secondaryKeyCode = key;
    updateKeyButtonText(ui->secondaryKeyButton, key);
}

void SettingsDialog::keyPressEvent(QKeyEvent *event)
{
    // Если ждем нажатия клавиши для бинда
    if (isWaitingForKey && activeKeyButton && !event->isAutoRepeat()) {
        int key = event->key();

        // Игнорируем модификаторы как отдельные клавиши
        if (key == Qt::Key_Shift || key == Qt::Key_Control ||
            key == Qt::Key_Alt || key == Qt::Key_Meta ||
            key == Qt::Key_AltGr) {
            return;
        }

        // ОСОБАЯ ОБРАБОТКА ПРОБЕЛА
        if (key == Qt::Key_Space) {
            // Для пробела нужно дополнительное действие
            if (activeKeyButton == ui->primaryKeyButton) {
                primaryKeyCode = Qt::Key_Space;
                updateKeyButtonText(ui->primaryKeyButton, Qt::Key_Space);
            } else if (activeKeyButton == ui->secondaryKeyButton) {
                secondaryKeyCode = Qt::Key_Space;
                updateKeyButtonText(ui->secondaryKeyButton, Qt::Key_Space);
            }

            isWaitingForKey = false;
            activeKeyButton = nullptr;

            // Восстанавливаем фокус после пробела
            QTimer::singleShot(100, this, [this]() {
                setFocus();
            });

            event->accept();
            return;
        }

        // Обработка других клавиш
        if (activeKeyButton == ui->primaryKeyButton) {
            primaryKeyCode = key;
            updateKeyButtonText(ui->primaryKeyButton, key);
        } else if (activeKeyButton == ui->secondaryKeyButton) {
            secondaryKeyCode = key;
            updateKeyButtonText(ui->secondaryKeyButton, key);
        }

        isWaitingForKey = false;
        activeKeyButton = nullptr;
        event->accept();
        return;
    }

    // Если не в режиме бинда, обрабатываем Escape для отмены
    if (event->key() == Qt::Key_Escape && isWaitingForKey) {
        isWaitingForKey = false;
    }
}

```

```

        if (activeKeyButton == ui->primaryKeyButton) {
            updateKeyButtonText(ui->primaryKeyButton, primaryKeyCode);
        } else if (activeKeyButton == ui->secondaryKeyButton) {
            updateKeyButtonText(ui->secondaryKeyButton, secondaryKeyCode);
        }
        activeKeyButton = nullptr;
        event->accept();
        return;
    }

    QDialog::keyPressEvent(event);
}

void SettingsDialog::onButtonBoxClicked(QAbstractButton *button)
{
    QDialogButtonBox::StandardButton stdButton = ui->buttonBox->standardButton(button);

    switch (stdButton) {
    case QDialogButtonBox::Ok:
        // Применяем настройки и закрываем
        emit settingsChanged();
        accept();
        break;

    case QDialogButtonBox::Apply:
        // Только применяем настройки, не закрываем
        emit settingsChanged();
        break;

    case QDialogButtonBox::Cancel:
        // Просто закрываем без применения
        reject();
        break;

    default:
        // Обработка других кнопок, если есть
        break;
    }
}

void SettingsDialog::onPrimaryKeyButtonClicked()
{
    isWaitingForKey = true;
    activeKeyButton = ui->primaryKeyButton;
    ui->primaryKeyButton->setText("Нажмите клавишу...");
    ui->primaryKeyButton->setFocus();

    // Временно меняем политику фокуса для захвата пробела
    ui->primaryKeyButton->setFocusPolicy(Qt::StrongFocus);
    grabKeyboard(); // Захватываем все события клавиатуры
}

void SettingsDialog::onSecondaryKeyButtonClicked()
{
    isWaitingForKey = true;
    activeKeyButton = ui->secondaryKeyButton;
    ui->secondaryKeyButton->setText("Нажмите клавишу...");
    ui->secondaryKeyButton->setFocus();

    ui->secondaryKeyButton->setFocusPolicy(Qt::StrongFocus);
    grabKeyboard(); // Захватываем все события клавиатуры
}

```

```

}

void SettingsDialog::updateKeyButtonText(QPushButton *button, int key)
{
    button->setText(keyToString(key));

    // Восстанавливаем нормальное состояние кнопки
    if (button == activeKeyButton) {
        releaseKeyboard(); // Освобождаем захват клавиатуры
        activeKeyButton = nullptr;
        isWaitingForKey = false;
    }

    // Восстанавливаем нормальную политику фокуса
    button->setFocusPolicy(Qt::TabFocus);
}

QString SettingsDialog::keyToString(int key)
{
    switch (key) {
        case Qt::Key_Space: return "Пробел";
        case Qt::Key_X: return "X";
        case Qt::Key_Z: return "Z";
        case Qt::Key_A: return "A";
        case Qt::Key_S: return "S";
        case Qt::Key_D: return "D";
        case Qt::Key_F: return "F";
        case Qt::Key_J: return "J";
        case Qt::Key_K: return "K";
        case Qt::Key_L: return "L";
        case Qt::Key_Semicolon: return ";";
        case Qt::Key_Enter:
        case Qt::Key_Return: return "Enter";
        case Qt::Key_Up: return "Стрелка вверх";
        case Qt::Key_Down: return "Стрелка вниз";
        case Qt::Key_Left: return "Стрелка влево";
        case Qt::Key_Right: return "Стрелка вправо";
        case Qt::Key_Tab: return "Tab";
        case Qt::Key_Escape: return "Esc";
        case Qt::Key_Backspace: return "Backspace";
        case Qt::Key_CapsLock: return "Caps Lock";
        case Qt::Key_Delete: return "Delete";
        case Qt::Key_Home: return "Home";
        case Qt::Key_End: return "End";
        case Qt::Key_PageUp: return "Page Up";
        case Qt::Key_PageDown: return "Page Down";
        default:
            QString text = QKeySequence(key).toString();
            if (text.isEmpty()) {
                return QString("Key %1").arg(key);
            }
            return text;
    }
}

```

settingsdialog.h

```

#ifndef SETTINGSDIALOG_H
#define SETTINGSDIALOG_H

#include <QDialog>
#include <QKeySequenceEdit>
#include <qabstractbutton.h>

```



```

namespace Ui {
class SettingsDialog;
}

class SettingsDialog : public QDialog
{
    Q_OBJECT

public:
    explicit SettingsDialog(QWidget *parent = nullptr);
    ~SettingsDialog();

    float hitVolume() const;
    float missVolume() const;
    float musicVolume() const;
    int primaryKey() const;
    int secondaryKey() const;

    void setHitVolume(float volume);
    void setMissVolume(float volume);
    void setMusicVolume(float volume);
    void setPrimaryKey(int key);
    void setSecondaryKey(int key);

signals:
    void settingsChanged();

protected:
    void keyPressEvent(QKeyEvent *event) override;

private slots:
    void onPrimaryKeyButtonClicked();
    void onSecondaryKeyButtonClicked();

    // Убираем старые слоты, добавляем новый
    void onIconButtonClicked(QAbstractButton *button);

private:
    Ui::SettingsDialog *ui;
    bool isWaitingForKey = false;
    QPushButton *activeKeyButton = nullptr;

    int primaryKeyCode = Qt::Key_Space;
    int secondaryKeyCode = Qt::Key_X;

    void updateKeyButtonText(QPushButton *button, int key);
    QString keyToString(int key);
};

#endif // SETTINGSDIALOG_H

```

